CHAPTER VIII

Introduction to Apache Spark

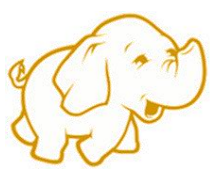
1. INTRODUCTION TO APACHE SPARK
2. SPARK ARCHITECTURE
3. SPARK PROGRAMMING CONCEPTS
4. SPARK PROGRAMMING

1. Understand the role and motivation of Apache Spark in modern data processing. — Explain the limitations of Hadoop MapReduce. Identify Spark's key advantages: speed, in-memory computation, fault tolerance, and versatility.
2. Describe the core architecture of Spark and how it manages distributed processing — Manager, and DAG Scheduler. Distinguish between narrow and wide transformations and understand the concept of
3. Apply fundamental Spark programming concepts to build and run distributed applications. — Use RDDs and DataFrames to perform basic transformations and actions. Write and interpret simple Spark programs in Scala or Python

Section N° 1

Introduction to Apache Spark

MapReduce	❑ Easily writing applications to process vast amounts of data in parallel on large clusters in a reliable, fault-tolerant manner ❑ Takes care of scheduling tasks, monitoring them and re-executes the failed tasks
HDFS & MapReduce	❑ Running on the same set of nodes ❑ Compute nodes and storage nodes same (keeping data close to the computation)
YARN & MapReduce	❑ A single master resource manager, ❑ One slave node manager per node, and AppMaster per application



MapReduce | Other Data Processing Frameworks

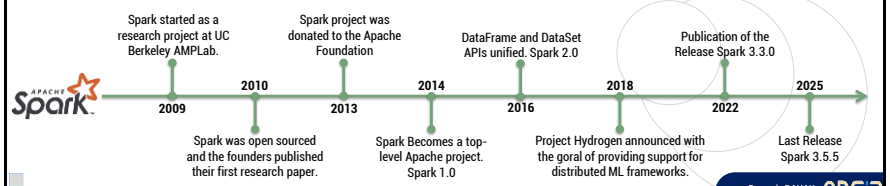
YARN
Resource Management

HDFS
Distributed File Storage

Introduction to Apache Spark
What is Spark?

Section 01 — 5

- Apache Spark is an open-source, distributed processing system used for big data analytics.
  - It provides development APIs in Java, Scala, Python and R,
  - Supports code reuse across multiple nodes
  - Batch processing, interactive queries, real-time analytics, ML, and graph processing.
- It utilizes in-memory caching, and optimized query execution for fast analytic queries against data of any size.
  - Initially started by Zaharia at UC Berkeley's AMPLab in 2009, open sourced in 2010.
  - Donated to the Apache Foundation in 2013, and switched its license to Apache 2.0 from BSD.
  - In February 2014, Spark became a Top-Level Apache Project
- Spark has its own cluster management computation, it uses Hadoop for storage purpose only

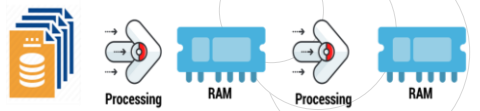


Introduction to Apache Spark
Spark Vs Map Reduce

Section 01 — 6

MapReduce need a Stable Intermediate and Distributed storage like HDFS comprising repetitive computations with data replications and data serialization, which made the process a lot slower.

RDDs (distributed collection of memory with permissions as Read-only) are easy to use and effortless to create as data is imported from data sources and dropped into RDDs. Further, the operations are applied to process them.



Introduction to Apache Spark
Spark Vs Map Reduce

Section 01 — 7

Aspect	MapReduce	Apache Spark
1. Processing Model	Batch processing only	Batch + Real-time (with Spark Streaming)
2. Execution Model	Disk-based I/O between map and reduce stages	In-memory computation using RDDs
3. Performance	Slower due to frequent disk reads/writes	Up to 100x faster with in-memory processing
4. Ease of Use	Complex, verbose Java code	Concise APIs in Scala, Java, Python, R, and SQL
5. Iterative Algorithms	Inefficient; each iteration requires a new job	Efficient; keeps data in memory across iterations
6. Data Structures	Key-value pairs only	RDDs, DataFrames, and Datasets
7. Streaming Support	Not supported natively	Spark Streaming/Structured Streaming
8. Machine Learning	External tools like Mahout	Built-in MLlib library
9. Execution Engines	YARN only	YARN, Mesos, Kubernetes, Standalone
10. Fault Tolerance	Yes, via HDFS replication	Yes, via RDD lineage and DAG recovery

Introduction to Apache Spark
Features of Spark

Section 01 — 8

Swift Processing
Spark offers high data processing speed. That is about 100x faster in memory and 10x faster on the disk.

Dynamic in Nature
Basically, it is possible to develop a parallel application. Since there are 80 high-level operators available in Spark.

In-Memory Computation
The increase in processing speed is possible due to in-memory processing.

Reusability
We can easily reuse code for batch processing or join stream against historical data. Also to run ad hoc queries on stream state.

Spark Fault Tolerance
Spark offers fault tolerance. It is possible through Spark's core abstraction-RDD.

Real-Time Stream Processing
Basically, Hadoop does not support real-time processing, what is supported in Spark.

Lazy Evaluation in Spark
All the transformations we make in Spark RDD are Lazy in nature that is it does not give the result right away rather a new RDD is formed from the existing one.

Polyglot
Spark provides high-level APIs in Java, Scala, Python, and R. It also provides a shell in Scala and Python.

Support for Sophisticated Analysis
There are dedicated tools in Spark. Such as for streaming data interactive/declarative queries, machine learning which add-on to map and reduce.

Integrated with Hadoop
As we know Spark is flexible. It can run independently and also on Hadoop YARN Cluster Manager. Even it can read existing Hadoop data.

Spark GraphX
In Spark, a component for graph and graph parallel computation, we have GraphX

Cost Efficient
For Big data problem as in Hadoop, a large amount of storage and the large data centre is required during replication. Hence, Spark programming turns out to be a cost-effective solution

Introduction to Apache Spark
Spark Use Cases

Data integration

- The data generated by systems are not consistent enough to combine for analysis.
- To fetch consistent data from systems we can use processes like Extract, transform, and load (ETL).
- Spark is used to reduce the cost and time required for this ETL process.

Stream processing

- It is always difficult to handle the real-time generated data such as log files.
- Spark is capable enough to operate streams of data and refuses potentially fraudulent operations.

Machine learning

- Machine learning approaches become more feasible and increasingly accurate due to enhancement in the volume of data.
- As spark is capable of storing data in memory and can run repeated queries quickly, it makes it easy to work on machine learning algorithms.

Interactive analytics

- Spark is able to generate the respond rapidly.
- So, instead of running pre-defined queries, we can handle the data interactively.

Introduction to Apache Spark
Spark Use Cases

E-Commerce(Alibaba & ebay)

Information about real time transaction can be passed to streaming clustering algorithms like alternating least squares or K-means clustering algorithm. The results can be combined with other data like social media profiles or product reviews to enhance the recommendations to customers based on new trends.

Health Care(MyFitnessPal)

MyFitnessPal helps people achieve a healthy lifestyle through better diet and exercise. MyFitnessPal uses apache spark to clean the data entered by users with the end goal of identifying high quality food items.

Gaming(Tencent, Riot)

Spark is used in the gaming industry to identify patterns from the real-time in-game events and respond to them to harvest lucrative business opportunities like targeted advertising, auto adjustment of gaming levels based on complexity, player retention and many more.

Media

Yahoo uses Apache Spark for personalizing its news webpages and for targeted advertising. It uses ML algorithms that run on Spark to find out what kind of news users are interested to read. Netflix uses Spark for real-time stream processing to provide online recommendations to its customers.

Section N° 2

Spark Architecture

Spark Architecture
Spark Architecture

Apache Spark Architecture is based on two main abstractions:

- Resilient Distributed Datasets (RDD)
- Directed Acyclic Graph (DAG)

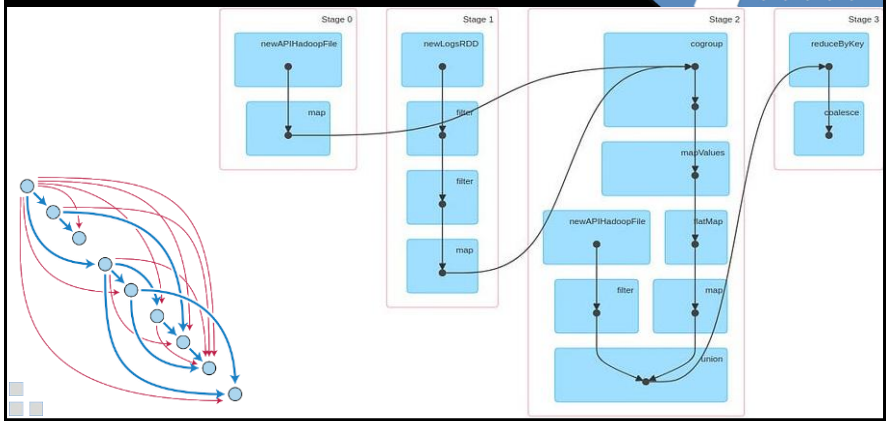
RDDs are the building blocks of any Spark application

- Resilient: Fault tolerant and is capable of rebuilding data on failure
- Distributed: Distributed data among the multiple nodes in a cluster
- Dataset: Collection of partitioned data with values

Directed Acyclic Graph (DAG)

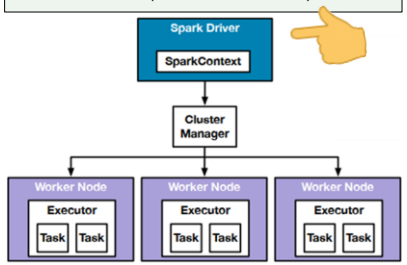
- Direct: Transformation is an action which transitions data partition state from A to B.
- Acyclic: Transformation cannot return to the older partition
- DAG is a sequence of computations performed on data where each node is an RDD partition and edge is a transformation on top of data.

Spark Architecture
Directed Acyclic Graph (DAG) Example



Spark Architecture
Spark Architecture

Apache Spark follows master/slave architecture with two main daemons and a cluster manager –
- ❑ Master Daemon – (Master/Driver Process)
- ❑ Worker Daemon –(Slave Process/Executor)



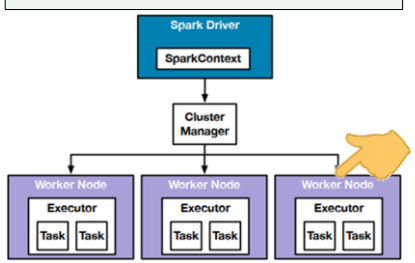
A spark cluster has a single Master and any number of Slaves/Workers

Spark Driver
Master Node of a Spark Application(Master)

- ❑ It is the central point and the entry point of the Spark Shell (Scala, Python, and R).
- ❑ The driver program runs the main () function of the application and is the place where the Spark Context is created. It is similar to your database connection.
- ❑ Spark Driver contains various components
  - – DAGScheduler,
  - – TaskScheduler,
  - – BackendScheduler and
  - – BlockManager

Spark Architecture
Spark Architecture

Apache Spark follows master/slave architecture with two main daemons and a cluster manager –
- ❑ Master Daemon – (Master/Driver Process)
- ❑ Worker Daemon –(Slave Process/Executor)



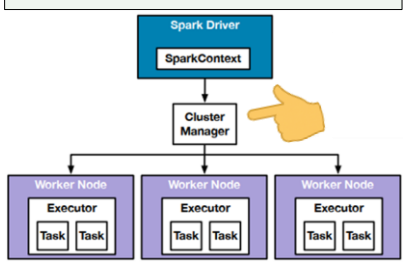
A spark cluster has a single Master and any number of Slaves/Workers

Executor/Worker Node(Slave)

- ❑ Executor is a distributed agent responsible for the execution of tasks.
- ❑ Every spark applications has its own executor process.
- ❑ Executors usually run for the entire lifetime of a Spark application and this phenomenon is known as —Static Allocation of Executors. However, users can also opt for dynamic allocations of executors wherein they can add or remove spark executors dynamically to match with the overall workload.
- ❑ Executor performs all the data processing.
- ❑ Reads from and Writes data to external sources.
- ❑ Executor stores the computation results data in-memory, cache or on hard disk drives.
- ❑ Interacts with the storage systems.

Spark Architecture
Spark Architecture

Apache Spark follows master/slave architecture with two main daemons and a cluster manager –
- ❑ Master Daemon – (Master/Driver Process)
- ❑ Worker Daemon –(Slave Process/Executor)



A spark cluster has a single Master and any number of Slaves/Workers

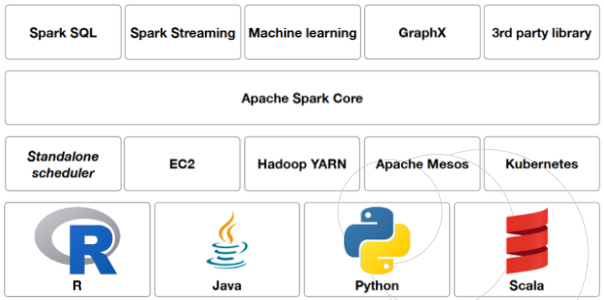
Cluster Manager

- ❑ An external service responsible for acquiring resources on the spark cluster and allocating them to a spark job.
- ❑ A Spark application can leverage for the allocation and deallocation of various physical resources such as memory for client spark jobs, CPU memory, etc.
- ❑ Hadoop YARN, Apache Mesos or the simple standalone spark cluster manager either of them can be launched on-premise or in the cloud for a spark application to run.

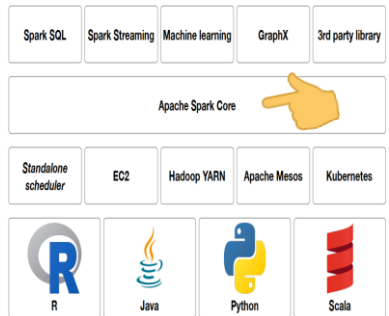
Spark Architecture
Spark Components

The Spark project contains multiple closely integrated components.

At its core, Spark is a computational engine that is responsible for scheduling, distributing, and monitoring applications consisting of many computational tasks across many worker machines, or a computing cluster.



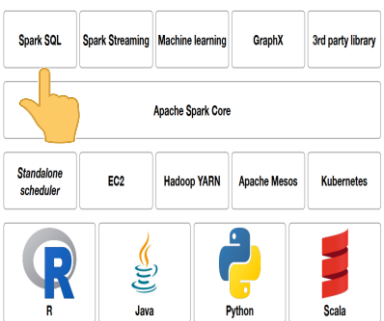
Spark Architecture
Spark Components



Spark Core

- Spark Core contains the basic functionality of Spark, including components for task scheduling, memory management, fault recovery, interacting with storage systems, and more.
- Spark Core is also home to the API that defines resilient distributed datasets (RDDs), which are Spark's main programming abstraction.
- Spark Core provides many APIs for building and manipulating these collections.

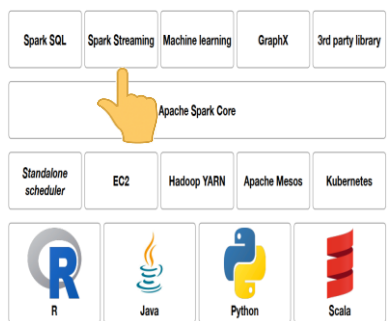
Spark Architecture
Spark Components



Spark SQL

- Spark SQL is Spark's package for working with structured data. It allows querying data via SQL as well as the Apache Hive variant of SQL (HQL) and it supports many sources of data, including Hive tables and JSON.
- Spark SQL allows developers to intermix SQL queries with the programmatic data manipulations supported by RDDs in Python,
- Java, and Scala, all within a single application, thus combining SQL with complex analytics.

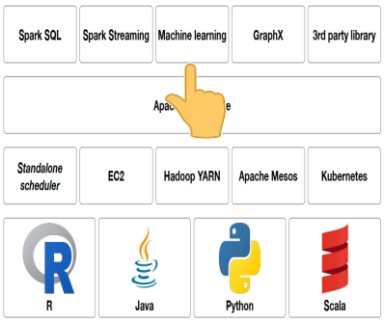
Spark Architecture
Spark Components



Spark Streaming

- Spark Streaming is a Spark component that enables processing of live streams of data. Eg. logfiles generated by production web servers.
- Spark Streaming was designed to provide the same degree of fault tolerance, throughput, and scalability as Spark Core.

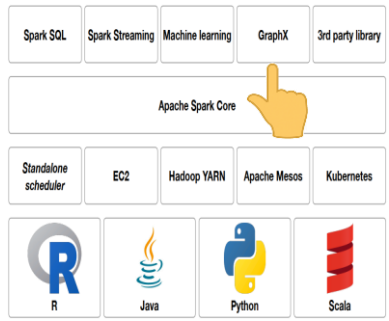
Spark Architecture
Spark Components



Mllib

- Spark comes with a library containing common machine learning (ML) functionality, called MLlib.
- MLlib provides multiple types of machine learning algorithms, including classification, regression, clustering, and collaborative filtering, as well as supporting functionality such as model evaluation and data import.

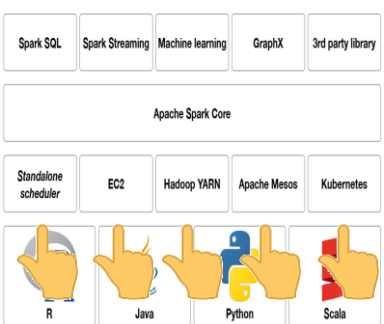
Spark Architecture
Spark Components



GraphX

- GraphX is a library for manipulating graphs (e.g., a social network's friend graph) and performing graph-parallel computations.
- Like Spark Streaming and Spark SQL, GraphX extends the Spark RDD API, allowing us to create a directed graph with arbitrary properties attached to each vertex and edge.
- GraphX also provides various operators for manipulating graphs (e.g., subgraph and mapVertices) and a library of common graph algorithms (e.g., PageRank and triangle counting).

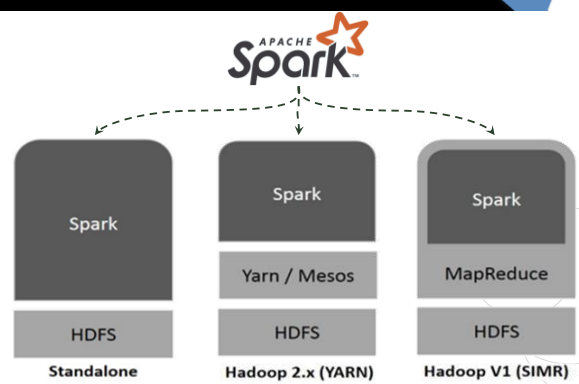
Spark Architecture
Spark Components



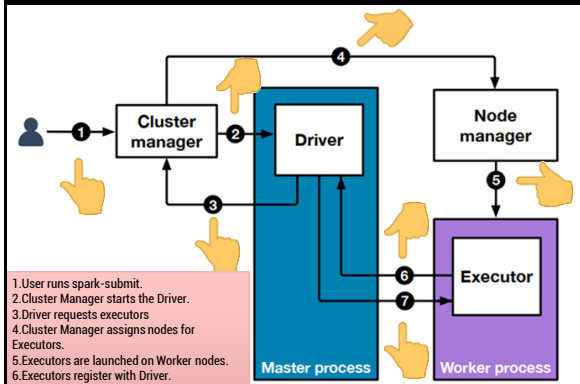
Cluster Managers

- Spark is designed to efficiently scale up from one to many thousands of compute nodes.
- To achieve this while maximizing flexibility, Spark can run over a variety of cluster managers, including Hadoop YARN, Apache Mesos, and a simple cluster manager included in Spark itself called the Standalone Scheduler.

Spark Architecture
Spark Execution Modes



Spark Architecture
Launching a Spark Application



1.User runs spark-submit.
2.Cluster Manager starts the Driver.
3.Driver requests executors
4.Cluster Manager assigns nodes for Executors.
5.Executors are launched on Worker nodes.
6.Executors register with Driver.
7.Driver sends tasks to Executors to begin execution.

2 Launch the Driver (Cluster Manager → Driver)

- The Cluster Manager (YARN, Mesos, Standalone, or Kubernetes) allocates a container or process and launches the Driver program.
- The Driver is the central coordinator of a Spark application.

Section N° 3

Spark Programming Concepts

Spark Programming Concepts
Resilient Distributed Datasets (RDD)

It is a fundamental data structure of Spark.

It is an immutable distributed collection of objects.

Each dataset in RDD is divided into logical partitions, which may be computed on different nodes of the cluster.

Formally, it is a read-only, partitioned collection of records.

It can be created through deterministic operations on either data on stable storage or other RDDs.

It is a fault-tolerant collection of elements that can be operated on in parallel

CODE : SCALA

```
1. val rdd = spark.sparkContext.parallelize(Seq(1,2,3,4,5))
2. val squaredRDD = rdd.map(x => x * x)
3. squaredRDD.collect() // Output: Array(1, 4, 9, 16, 25)
```

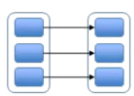
RDDs support two types of operations:
Transformations
Actions

RDDs are lazy – they don't compute anything until an action is called.

Spark Programming Concepts
Narrow Vs Wide Transformations

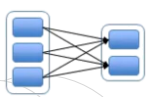
How data moves between partitions during transformations?

Narrow Transformations



Each partition of the parent RDD / DataFrame is used by at most one partition of the child.
- ➡ No shuffle across the network
- ➡ Fast, pipelined execution

Wide Transformations



Each partition of the child RDD / DataFrame may depend on many partitions of the parent.
- ➡ Requires shuffling of between workers
- ➡ More expensive (I/O, network, barrier between stages)

Feature	Narrow Transformation	Wide Transformation
Data Movement	Local	Shuffle across network
Execution Stage	Same stage	New stage required
Performance	Fast	Slower due to I/O
Fault Tolerance	Simple	Complex, lineage-based recovery
Examples (RDD)	map, filter, flatMap	reduceByKey, join, groupBy
Examples (DataFrame)	select, filter, withColumn	join, groupBy, orderBy

Spark Programming Concepts
RDD Operations : Transformations

Narrow Transformations		
map(func)	Applies function to each element	rdd.map(x => x * 2)
flatMap(func)	Like map, but can return multiple items	rdd.flatMap(line => line.split(" "))
filter(func)	Keeps elements matching condition	rdd.filter(x => x > 10)
mapPartitions(func)	Applies function to each partition	rdd.mapPartitions(iter => iter.map(_ * 2))
sample(...)	Returns a sampled subset of the RDD	rdd.sample(false, 0.1)
union(otherRDD)	Combines two RDDs (same partitioning)	rdd1.union(rdd2)
glom()	Converts each partition into an array	rdd.glom()

Wide Transformations		
groupByKey()	Groups values with the same key	rdd.groupByKey()
reduceByKey()	Aggregates values for each key	rdd.reduceByKey(_ + _)
join(otherRDD)	Joins two RDDs by key	rdd1.join(rdd2)
leftOuterJoin()	Left outer join by key	rdd1.leftOuterJoin(rdd2)
rightOuterJoin()	Right outer join	rdd1.rightOuterJoin(rdd2)

Wide Transformations		
distinct()	Removes duplicates	rdd.distinct()
repartition(n)	Changes the number of partitions	rdd.repartition(4)
coalesce(n)	Reduce partitions	rdd.coalesce(2)
sortBy()	Sorts the RDD based on a key	rdd.sortBy(x => x)
cogroup()	Groups RDDs with same keys	rdd1.cogroup(rdd2)

Spark Programming Concepts
RDD Operations : Actions

Action	Description	Example (Scala)
collect()	Returns all elements of the RDD to the driver (use cautiously).	val data = rdd.collect()
count()	Returns the number of elements in the RDD.	val total = rdd.count()
first()	Returns the first element of the RDD.	val item = rdd.first()
take(n)	Returns the first n elements as an array.	val top5 = rdd.take(5)
takeOrdered(n)	Returns the first n elements after sorting (ascending order).	val smallest = rdd.takeOrdered(3)
top(n)	Returns the top n elements (descending order).	val largest = rdd.top(3)
reduce(func)	Aggregates the elements using a binary function.	val sum = rdd.reduce((a, b) => a + b)
fold(zero)(func)	Like reduce but with a starting "zero" value.	val sum = rdd.fold(0)(_ + _)
aggregate(z)(s, c)	Combines elements using two functions: one within and one across partitions.	See complex example above.
foreach(func)	Applies a function to each element (used for side-effects, like printing).	rdd.foreach(x => println(x))
countByValue()	Counts the occurrences of each unique value in the RDD.	val counts = rdd.countByValue()
saveAsTextFile()	Saves the RDD as a text file (HDFS or local path).	rdd.saveAsTextFile("output/")
saveAsSequenceFile()	Saves key-value RDDs as Hadoop SequenceFiles.	rdd.saveAsSequenceFile("seqoutput/")
saveAsObjectFile()	Saves the RDD using Java serialization.	rdd.saveAsObjectFile("objoutput/")
lookup(key)	For PairRDDs: returns all values associated with a specific key.	rdd.lookup("key")

Spark Programming Concepts
Spark DataFrames (DF)

Built on top of RDDs
DataFrames = (RDDs + Schema)

Higher-level data structure and API in Spark
Supports SQL queries and high-level transformations

Implemented using an immutable distributed table of records with rows, columns and a schema

Divided in partitions, distributed across multiple executors

Analogous to:
- ❑ a Table in a DB (but: no indexes, primary keys, constraints, etc...)
- ❑ a DataFrame in Python / R

Offers better performance and readability than RDDs.

CODE : SCALA

```
1. df=spark.read.csv("employees.csv",header=True,inferSchema=True)
2. df.select("id","name").filter("gender"="MALE").show()
```

DFs support two types of operations:
- ❑ Transformations
- ❑ Actions

DFs are lazy — they don't compute anything until an action is called.

Spark Programming Concepts
DataFrame Operations : Transformations

Narrow Transformations		
select(...)	Selects specific columns	df.select("name", "age")
withColumn(...)	Adds/replaces a column using an expression	df.withColumn("bonus", $"salary" * 0.1)
drop(...)	Removes a column	df.drop("age")
filter(...) / where(...)	Filters rows by condition	df.filter($"age" > 30)
limit(n)	Returns the first n rows	df.limit(5)
na.fill(...)	Replaces nulls with specified values	df.na.fill(0)

Wide Transformations		
groupBy(...)	Groups rows by key(s)	df.groupBy("dept")
agg(...)	Aggregates grouped data	df.groupBy("dept").agg(avg("salary"))
orderBy(...), sort(...)	Global sorting — needs shuffle	df.orderBy($"salary".desc)
join(...)	Combines rows from two DataFrames (by key)	df1.join(df2, "id")

Narrow Transformations		
na.drop()	Drops rows with nulls	df.na.drop()
dropDuplicates(...)	Removes duplicates **without** causing shuffle (1 partition)	df.dropDuplicates("name")*
cache(), persist()	Marks for caching/persisting in memory or disk	df.cache()
coalesce(n)	Reduces number of partitions **without shuffle**	df.coalesce(2)

Wide Transformations		
union(...)	Combines two DataFrames (same schema)	df1.union(df2)
intersect(...)	Returns rows common to both DataFrames	df1.intersect(df2)
except(...)	Returns rows in df1 not in df2	df1.except(df2)
dropDuplicates(...)*	Wide if Spark needs to **shuffle to check uniqueness**	df.dropDuplicates()
repartition(n)	Increases number of partitions (involves shuffle)	df.repartition(4)

Spark Programming Concepts
DataFrame Operations : Actions

Action	Description	Example (Scala)
show(n)	Displays the first n rows in tabular form (default: 20).	df.show(10)
collect()	Returns **all rows** as an array to the driver.	val data = df.collect()
take(n)	Returns the first n rows as an array.	val top5 = df.take(5)
head()	Returns the first row (or array of first n rows).	val first = df.head()
first()	Returns the very first row.	val firstRow = df.first()
count()	Returns the number of rows in the DataFrame.	val total = df.count()
reduce(func)	Aggregates the rows using a binary function (usually used with Row).	df.rdd.reduce((a, b) => ...)
foreach(func)	Runs a function on each row (side effects only, like printing/logging).	df.foreach(row => println(row))
foreachPartition(func)	Runs a function on each partition (more efficient than foreach).	df.foreachPartition(part => ...)
toLocalIterator()	Returns an iterator that pulls rows one at a time to the driver.	val it = df.toLocalIterator()
write	Saves the DataFrame to files (CSV, Parquet, etc.).	df.write.csv("output/")
write.format(...).save()	Generalized write method with format and options.	df.write.format("parquet").save("out")
rdd	Converts DataFrame to its underlying RDD.	val rdd = df.rdd

Section N° 4

Spark Programming

Spark Programming Concepts
Examples

CODE : SCALA

```scala
val employees = spark.read
  .option("header", "true")
  .option("inferSchema", "true")
  .csv("/shared/employees.csv")

val contributions = spark.read
  .option("header", "true")
  .option("inferSchema", "true")
  .csv("/shared/contributions.csv")

val reimbursements = spark.read
  .option("header", "true")
  .option("inferSchema", "true")
  .csv("/shared/reimbursements.csv")
```


Spark Programming Concepts
Examples

CODE : SCALA

```scala
val females = employees.filter($"gender" === "FEMALE")
females.show()
```

CODE : SCALA

```scala
val empWithContrib = employees
  .join(contributions, employees("id") === contributions("employee_id"),
"left")
  .select("id", "name", "amount")
empWithContrib.show()
```

Spark Programming Concepts
Examples

CODE : SCALA

```scala
val totalContrib = contributions
  .groupBy("employee_id")
  .agg(sum("amount").alias("total_contrib"))
totalContrib.show()
```

CODE : SCALA

```scala
val empWithReimb = employees
  .filter($"gender" === "FEMALE")
  .join(reimbursements, employees("id") === reimbursements("employee_id"),
"left_anti")
empWithReimb.show()
```


Spark Programming Concepts
Examples

CODE : SCALA

```scala
val totalReimb = reimbursements
  .groupBy("employee_id")
  .agg(sum("amount").alias("total_reimb"))

val summary = employees
  .join(totalContrib, employees("id") === totalContrib("employee_id"),
"left")
  .join(totalReimb, employees("id") === totalReimb("employee_id"), "left")
  .select($"id", $"name", $"gender", $"total_contrib", $"total_reimb")

summary.show()
```


Chapter :

Thank You

Q & A